

Fiche de révision ultime — Une vie de fourmi

Tout ce qu'il faut savoir pour défendre le projet en 5 minutes et répondre aux questions

Équipe : Romain · Lisa · Yannis

Pitch en 20 secondes. La fourmilière est un graphe. Comme des dizaines de fourmis circulent en même temps dans des salles à capacité limitée, un plus court chemin ne suffit pas. On déplie le graphe dans le temps et on calcule un flot maximal : le premier nombre d'étapes T qui laisse passer les F fourmis est le minimum.

1. Les règles du sujet (à ne jamais modifier)

1. Toutes les fourmis partent de S_v et doivent toutes finir dans S_d .
2. À chaque étape, une fourmi attend ou va dans une salle voisine.
3. Une salle intermédiaire contient 1 fourmi par défaut ; $SN\{X\}$ signifie capacité X .
4. S_v et S_d ne sont pas limités ; les tunnels non plus.
5. Les déplacements sont simultanés : une fourmi peut entrer dans une salle au moment où l'occupante la quitte.
6. Objectif : faire arriver toutes les fourmis en un minimum d'étapes (l'arrivée de la dernière compte).

2. L'algorithme en 7 étapes

1. **Lecture du fichier** : $f=50$, $S_1 \{ 5 \}$, $S_v - S_1...$ Espaces, lignes vides, BOM et CRLF acceptés ; une ligne invalide donne une erreur avec son numéro.
2. **BFS** (`ants.bfs_distances`) : distance minimale d de S_v à S_d . Personne n'arrive avant d étapes : c'est la borne basse.
3. **Graphe temporel** : pour un horizon T , une copie ($salle, t$) de chaque salle pour $t = 0 \dots T$. Attendre = $(a, t) \rightarrow (a, t+1)$; se déplacer = $(a, t) \rightarrow (b, t+1)$.
4. **Node splitting** : chaque ($salle, t$) devient entrée $\rightarrow$ sortie avec la capacité de la salle. C'est ce qui applique $SN\{X\}$.
5. **Max-flow (Edmonds-Karp)** : combien de fourmis peuvent arriver en T étapes ? Si c'est F , T est faisable.
6. **Recherche de T** : on teste $T = d, d+1, d+2...$ Le premier T faisable est l'optimum.
7. **Min-cost flow puis décomposition** : à T fixé, on choisit le planning le plus propre, puis on découpe le flot en trajets $f_1, f_2...$

3. Où est quoi ?

Notion	Où dans le code	Rôle
BFS	bfs_distances	borne basse d, layout, coûts
Graphe temporel	_build_time_expanded_graph	positions à chaque instant
Node splitting	arcs (salle, t, "in") → (salle, t, "out")	capacités $SN\{X\}$
Max-flow	max_ants_within (Edmonds-Karp de NetworkX)	F fourmis en T étapes ?
BFS dans Edmonds-Karp	interne à NetworkX	chemins augmentants dans le graphe résiduel
Min-cost flow	nx.min_cost_flow dans solve	trajets propres à T fixé
Décomposition	_decompose_flow	flot → trajets f1, f2...
Vérification	_validate_solution	toutes les règles, à chaque instant
Goulot	throughput_bottleneck (coupe minimale)	débit max par étape

4. Pourquoi CET algorithmme

Algorithme	Utilisé ?	Justification
BFS	oui	graphe non pondéré : BFS donne le plus court chemin (borne basse d)
Max-flow (Edmonds-Karp)	oui	seul outil qui répond « F fourmis simultanées en T étapes ? » avec les capacités
Graphe temporel	oui	transforme « position + temps » en un problème de flot statique
Min-cost flow	oui, après	seulement pour choisir un planning lisible, T ne bouge pas
DFS	non	ne garantit pas le plus court chemin, ne gère pas le trafic
Dijkstra	non	utile avec des poids ; ici tous les tunnels valent 1
Floyd-Warshall	non	toutes les paires de distances : inutile, on part de Sv

Piège classique. « Le plus court chemin suffit. » Non : sur fourmiere_cinq, $d = 5$ mais il faut 11 étapes, parce que les salles se remplissent. Le problème est un problème de trafic.

Chaque algorithme répond à un problème différent. BFS n'est pas « mauvais » : il trouve très bien un plus court chemin pour une fourmi, et on l'utilise. Mais BFS seul ne résout pas l'organisation simultanée de plusieurs fourmis avec des capacités de salles.

Le contre-exemple à connaître (inputs/demo/plus_court_chemin_vs_trafic.txt, 3 fourmis, capacité 1) : chemin A Sv → S1 → Sd (longueur 2) et chemin B Sv → S2 → S3 → Sd (longueur 3).

Stratégie	Étapes	Déroulé
Plus court chemin uniquement	4	E1 f1 Sv→S1 · E2 f1 S1→Sd, f2 Sv→S1 · E3 f2 S1→Sd, f3 Sv→S1 · E4 f3 S1→Sd
Notre solution (max-flow)	3	E1 f1 Sv→S1, f2 Sv→S2 · E2 f1 S1→Sd, f2 S2→S3, f3 Sv→S1 · E3 f2 S3→Sd, f3 S1→Sd

Plus court chemin individuel $\neq$ temps minimum pour toute la colonie. Visuel : outputs/demo_algorithmes/shortest_path_vs_flow.png.

Réponses flash

- « **Pourquoi ne pas simplement utiliser BFS ?** » On utilise justement BFS pour connaître la distance minimale entre le vestibule et le dortoir. Mais BFS trouve un chemin pour un trajet individuel. Notre problème demande de coordonner plusieurs fourmis en parallèle avec des capacités. Pour cela, on transforme le problème en réseau de flot temporel.

- « **Pourquoi pas Dijkstra ?** » Tous nos tunnels représentent exactement une étape, donc ils ont tous le même coût. Dijkstra donnerait la même distance que BFS en étant inutilement plus général. Et comme BFS, il ne gère pas à lui seul le trafic multi-fourmis.
- « **Pourquoi pas DFS ?** » DFS permet d'explorer le graphe, mais il ne garantit pas le plus court chemin et ne résout pas les contraintes de trafic. Nous n'avons donc aucune raison de l'utiliser ici.

5. Pourquoi le résultat est le minimum

- Avant d'étapes, c'est impossible (BFS).
- Pour un T donné, le graphe temporel contient **tous** les plannings autorisés par le sujet. Si le max-flow est inférieur à F, aucun planning en T étapes n'existe.
- On teste les T dans l'ordre croissant, donc le premier T faisable est le plus petit.
- La faisabilité est monotone : si T marche, T+1 marche aussi (les fourmis arrivées attendent dans Sd). Une recherche binaire serait donc correcte, mais le benchmark montre qu'elle est plus lente ici (elle teste de très grands T), donc on garde la recherche linéaire.
- **Preuve indépendante dans les tests** : une recherche exhaustive (BFS sur toutes les configurations de fourmis) donne le même minimum sur 400 fourmilières aléatoires. Elle détecte aussi 3 solveurs volontairement faux.

6. Le min-cost flow : des trajets propres

À T fixé, plusieurs plannings sont optimaux. On minimise, dans cet ordre strict :

1. la somme des dates d'arrivée (arrivées au plus tôt) : poids M^2 ;
2. le nombre de mouvements : poids M ;
3. les pas qui ne rapprochent pas de Sd : poids 1.

Avec $M = F \times T + 1$, une unité d'un objectif pèse plus que tous les objectifs suivants réunis. Sd est absorbant : une fourmi arrivée ne ressort pas.

Bug corrigé. Avant, un déplacement vers une salle plus lointaine pouvait coûter moins cher qu'une attente. Résultat : 75 allers-retours inutiles sur fourmiliere_3D et 188 sur salle_d_at-ant. Aujourd'hui : zéro, et environ 30 % de mouvements en moins, pour le même T.

7. Résultats sur les 9 fichiers officiels

Fichier	F	Salles	Tunnels	d	T optimal
fourmiliere_zero	2	2	4	2	2
fourmiliere_un	5	2	3	3	7
fourmiliere_deux	5	2	4	1	1
fourmiliere_trois	5	4	5	3	7
fourmiliere_quatre	10	6	9	5	9
fourmiliere_cinq	50	14	20	5	11
fourmiliere_3D	50	9	15	4	14
La_hormiguera_de_la_muerte	30	10	46	4	9
salle_d_at-ant	100	21	28	6	15

- **fourmiliere_deux** : tunnel direct Sd - Sv, donc toutes les fourmis arrivent en 1 étape.
- **fourmiliere_un / trois** : un couloir de salles à capacité 1, donc une fourmi par étape : $d + F - 1 = 3 + 5 - 1 = 7$.
- **salle_d_at-ant** : les salles S4, S5, S15, S16 (capacité 1) freinent ; une partie des fourmis fait un détour par S21 pour rejoindre les autres branches.

- Tableau complet et temps d'exécution : `outputs/summary/results.md`.

8. Les visualisations

- `graphe.png` : le graphe complet, avec la capacité de chaque salle.
- `etapes/etape_XX.png` et `animation.gif` : chaque étape sur le graphe complet. Orange = déplacements de l'étape (avec leur nombre) ; bleu clair = tunnels déjà empruntés ; gris = pas encore utilisés ; « 3 / 5 » = occupation / capacité ; cercle rouge = salle pleine.
- `flow_cumule.png` : le trafic total ; épaisseur proportionnelle au nombre de passages, flèche = sens, gris pointillé = tunnel jamais utilisé, cercle rouge = goulot (coupe minimale).
- Les positions sont identiques sur toutes les images (layout déterministe : Sv à gauche, Sd à droite, colonnes = distance BFS).

9. Les tests (`python -m unittest discover -s tests`)

Fichier	Ce qu'il vérifie
<code>test_parser.py</code>	syntaxes officielles, erreurs avec numéro de ligne, capacités contradictoires
<code>test_solver.py</code>	capacités, simultanéité, cul-de-sac, cycle, tunnel direct, $F = 100$, pas d'aller-retour, déterminisme
<code>test_officiels.py</code>	les 9 fichiers : règles respectées, T attendu, max-flow insuffisant à $T - 1$
<code>test_bruteforce.py</code>	400 + 150 cas aléatoires comparés à une recherche exhaustive
<code>test_contre_exemple.py</code>	plus court chemin seul : 4 étapes ; notre solveur : 3 étapes
<code>test_visualisation.py</code>	fichiers produits, nombre de frames du GIF, layout stable

10. Vulgarisation (paragraphe demandé par le sujet)

Imaginez la fourmilière comme un réseau de métro aux heures de pointe : les salles sont des stations, les tunnels des lignes, et certaines stations n'acceptent qu'un voyageur à la fois. Si tout le monde prend l'itinéraire le plus court, les stations saturent et chacun attend. Notre programme se demande d'abord : « peut-on faire rentrer toute la colonie en 5 étapes ? », puis 6, puis 7... Pour chaque durée, il essaie toutes les façons de répartir les fourmis entre les itinéraires, en respectant la place dans chaque salle. Dès que toute la colonie peut arriver, il s'arrête : cette durée est la plus courte possible, et il affiche le planning, fourmi par fourmi.

11. Les 5 phrases à retenir

1. Ce n'est pas un problème de chemin, c'est un problème de trafic.
2. Le BFS donne la borne basse ; le max-flow dit si F fourmis passent en T étapes.
3. La capacité d'une salle devient la capacité d'un arc (node splitting).
4. Le premier T faisable est le minimum, et une recherche exhaustive le confirme.
5. Le plus court chemin optimise une fourmi ; notre solution optimise toute la colonie.